Post-Pci-Clinical-Case-Retrieval-Drift

Tensium Samples

Pack identification

Category

CPU ML engineering

Pack / category name

Post-PCI Clinical Case Retrieval Drift

Why current models fail, and the capability this task builds

Even at a 100-step budget the runs fail: two of four now finish and submit, but the submitted terminal is wrong, and the other two exhaust the budget still searching. Given ample room, the model still cannot reconstruct the retriever's exact behavior. It stalls on a single inference: decoding the allocation rule, a query-byte phase with a top-k parity flip, from one audit receipt, then holding a correct multi-stage plan long enough to deploy it. The capability isolated is long-horizon specification inference: reconstructing exact system behavior from sparse evidence and sustaining it across dozens of dependent steps without drift.

1. Description

Description

This Category 02 task is a CPU-only ML engineering incident centered on a clinical case-retrieval pipeline used for pre-outcome review of hypotension after PCI. The agent is dropped into a broken but superficially plausible information-retrieval system and must diagnose and repair drift involving release authority, canonical serving identity, request-time eligibility, lexical ranking, rank fusion, partial-round allocation, and robustness to counterfactual changes. The visible retrieval panel and smoke tests are intentionally forgiving. Correctness is only established by driving the repaired implementation through a stateful operational sequence of inspections, audits, deployment snapshots, lossy validation, forced failure recovery, shadow checks, cutover, promotion, and final submission. The task matters because it evaluates whether an agent can repair and operate a real CPU-only ML/data pipeline under realistic release, provenance, temporal, ranking, and deployment constraints rather than merely produce a plausible one-shot code patch.

2. Task type & unit type

Task type

Hands-on long-horizon CPU ML engineering: inspect -> reason -> edit -> reproduce -> deploy -> validate -> diagnose -> recover -> revalidate -> audit -> shadow-test -> cut over -> promote -> submit

Unit type

One unit is one self-contained clinical retrieval incident. The unit contains a real 1,500-record PMC case-report corpus, a broken retriever, an agent-visible CLI and gated operation surface, release and provenance artifacts, hidden semantic counterfactuals, a forced runtime failure, and a deterministic verifier-owned final terminal. The required successful trajectory contains 27 accepted actions: 6 ordered observation actions and 21 ordered operational actions.

3. Verification method

Verification method

Verification is fully deterministic and does not use an LLM judge. The grader checks the complete ordered gate sequence, successful diagnosis and recovery of the forced stale_vector_cache failure, the final authoritative environment state, and terminal correctness. The deployed candidate is executed in an isolated subprocess against hidden real-corpus scenarios covering baseline, allocator, provenance, and landing-noise cases. Results, scores, eligible corpus counts, and domain counts are compared against independent reference implementations. The task receives 1.0 only if the gate walk, forced recovery, authoritative state, and terminal correctness all pass; otherwise it scores 0.0.

4. Technical domains / adjacent benchmarks

Technical domains

CPU-based information retrieval, clinical NLP, TF-IDF, word and character n-grams, rank fusion, temporal filtering, provenance handling, release management, Unicode normalization, and deterministic inference using scikit-learn-style methods.

Adjacent benchmarks

The task resembles Terminal-Bench and SWE-bench engineering tasks, but adds a stateful long-horizon operational workflow where the agent must inspect, repair, validate, recover from failure, and submit a verified terminal.

5. Human experts used

Human experts

The task was authored and calibrated. Review and QA feedback was supplied by the Tensium task-review team. No additional credential claims are made.

6. Raw base data

Raw base data

The task uses 1,500 real PMC case-report records, consisting of 500 cardiac, 500 vascular, and 500 systemic cases. The corpus includes PMCID, title, journal, final diagnosis, case summary, and related metadata. The workspace also contains a pre-outcome index case and operational artifacts in PDF, Word, HTML, JSON, CSV, and JSONL formats. The landing directory includes one unlisted duplicate delivery file, requiring the agent to establish release authority from the supplied evidence rather than file names alone.

7. What the world looks like

What the world looks like

The agent works inside /workspace/target/, with the broken retriever, real corpus, operational evidence, visible tests, and env_cli.py. It may edit only the candidate source code and has no internet access. The environment is CPU-only with 4 CPUs, 8 GB memory, no GPU, and a persistent backend service that maintains authoritative state and the gated incident lifecycle.

8. Time per task

Time per task

Approximately 8–10 authoring and QA hours per task.

9. QA process

QA process

QA runs through a systematic, automated pipeline rather than an ad-hoc pass. Deterministic controls mechanically screen the task for the failure modes that matter, reward-hacking and forgeable graders, answer leakage and reachable oracles, in-process grading, and false difficulty (for example, a "failure" that was only budget truncation). Each control is two-condition validated: it must fire on a planted defect and stay silent on a clean task. An adversarial control battery then executes cheat submissions against the real grader and requires every one to score zero, while a gold solution and a structurally different correct solution both score 1.0. An independent review pass grounds every finding in the actual task files, and difficulty is calibrated by running a frontier model at a fixed sampling standard inside the real isolated environment, with guards that separate genuine failures from budget/truncation artifacts.

QA includes an oracle solution, an alternate structurally different implementation, two independent reference engines, deterministic hidden scenarios, subprocess execution, isolation checks, timeout protection, and an anti-leak Docker build guard. Gate 0 successfully produced a 1.0 reward, confirming the reference solution can complete the task correctly. Full negative-control evidence should only be marked complete when the corresponding scored runs are available.

10. Difficulty calibration

SOTA model & pass@1

Claude Opus 5 was evaluated with four independent 100-step runs (pass@1: 0/4). Two runs finished and submitted a terminal that failed grading (genuine reasoning failures, at 77 and 78 steps); the other two exhausted the time and step budget (at 96 and 100 steps). No run reconstructed the correct behavior even with ample budget. This reflects the task's ultra-long-horizon structure, large 1,500-record dataset, and multi-stage investigation, repair, validation, recovery, and submission workflow.

Rollout records

All four Opus 5 rollouts scored reward 0.0 at a 100-step budget: two submitted an incorrect terminal (finish() called at 77 and 78 steps) and two exhausted the budget (96 and 100 steps). Gate 0 confirmed that the reference solution can complete the task successfully with a reward of 1.0. Frozen task package SHA-256: 25fe060d6cdb2dd55b5c2dfd6d96569fb487ace79c1bfdeda917e82821ee2429

11. Grading-design table

Component / gate / axis	What it measures	Weight / cap	How enforced
Environment-owned terminal	Whether the deployed retrieval implementation produces the correct hidden terminal values.	hard conjunction	Candidate executed in subprocess; hidden scenarios compared by value
Ordered gate walk	Completion of the required incident sequence	Hard requirement	Authoritative action history
Forced recovery	Correct diagnosis and recovery of stale_vector_cache	Hard requirement	Stateful broker and grader
Authoritative-state integrity	Clean validation, cutover and submission state	Hard requirement	Environment-owned state
Hidden terminal	Correct behavior across four scenario families	Hard requirement	Isolated subprocess replay
Isolation checks	Prevents candidate access to protected reference/runtime material	hard safety requirement	File permissions, agent-user isolation, reference-import/read probes

12. How the tasks were created

How created

The task was built around a real 1,500-record PMC clinical case corpus. The broken retriever was designed around interacting issues involving release provenance, serving identity, temporal eligibility, lexical ranking, rank fusion, and partial-round allocation. The visible smoke test is intentionally forgiving, while hidden counterfactual scenarios distinguish the correct implementation. A forced stale_vector_cache failure requires diagnosis and recovery before the agent can continue through validation, cutover, promotion, and submission. The reference solution repairs the retriever and drives the complete gated trajectory. Final grading evaluates the environment-owned deployed snapshot rather than trusting an agent-written answer.

---

13. QA checklist

☒ For hands-on tasks: a SECOND, structurally different correct solution also scores ~1.0 (grader accepts by behaviour, not gold's shape).

☒ Grader isolation: grader, answers, and reference solution live outside the agent's /workspace/target/ (in tests/ and solution/).

☒ For hands-on tasks: agent code runs in the verifier as a subprocess with a timeout - never imported into the grader.

☒ No answer / solution leakage: nothing in environment/workspace/ reveals the answer; reference deliverable absent; Dockerfile build guard enforces it.

☒ Deducibility: everything needed to solve is in the workspace; task is actually solvable; no invisible coin-flips.

☒ Real data, not synthetic: built on real OSS repos / public datasets / real experiment artifacts.

☒ Long-horizon by design: deciding signal is forced, results redirect the next step, no rewarded elective depth.

☒ Build hygiene: zip built in a temp/staging dir; solution/ is excluded from the agent's runtime workspace (present only for reviewer verification).

☒ Scrubbed: no secrets/keys, no platform job-ids or URLs, no internal project names or process jargon; no reproduced customer sample content.

☒ Realistic and professionally relevant: the task models a production clinical case-retrieval pipeline that drifts under release authority, as-of eligibility, canonical identity, rank fusion, and partial-round allocation, then must be repaired and driven through a gated deploy–validate–recover–promote–submit workflow.

☒ Clearly specified and internally consistent: the agent instructions, visible tests, gated operations sequence and deterministic grader agree on the required workflow.

☒ Reproducible and technically functional: the task uses pinned dependencies, deterministic seeds, frozen hidden probes and containerized verification.

☒ Distinct and non-duplicative: the package contains one self-contained task and no duplicate task variants.

☒ Licensed source material: PMC Open Access Subset; mixed Creative Commons (48% CC BY, 52% NC-restricted, 24% ND-restricted).

☒ Delivery format: the package follows the Harbor task structure with instruction, task metadata, environment, tests and solution directories.

14. Package overview & contents

This submission covers one complete CPU ML engineering task. The package includes the agent instructions, editable workspace, container environment, deterministic verifier, hidden evaluation scenarios, reference implementations, and solution drivers.

Field	Submission detail
Selected package	post-pci-clinical-case-retrieval-drift.zip
Task count	1 self-contained Harbor task
Task	cpu-ml-engineering/post-pci-clinical-case-retrieval-drift
Task type	CPU ML engineering; ultra-long-horizon investigation, repair, deployment, validation and recovery.
Unit / deliverable	One isolated task instance; corrected deployed retriever followed by successful validation, recovery, cutover, promotion and submission.
Verification	Binary deterministic grading using hidden real-corpus scenarios, the ordered gate walk, forced stale-cache recovery, authoritative state and isolated candidate execution.
Raw base data	PMC Open Access Subset (NCBI). All 1,500 records carry Creative Commons licenses, verified via the PMC OA service: 725 CC BY, 364 CC BY-NC, 363 CC BY-NC-ND, 47 CC BY-NC-SA, 1 CC BY-ND. Per-file row counts and SHA-256 digests are recorded in the data manifest. 774 records (51.6%) carry a non-commercial clause and 364 (24.3%) a no-derivatives clause; downstream use must respect these terms.

15. Per-task deep-dive

Task	What the task is about	Deliverable
Post-PCI Clinical Case Retrieval Drift	Diagnose and repair a clinical case retriever that drifts under release provenance, request-time eligibility, canonical identity, ranking, partial-round allocation and counterfactual changes. The agent must also diagnose and recover from a forced stale_vector_cache failure.	A corrected deployed retrieval pipeline that passes hidden real-corpus validation and completes the required recovery, shadow checks, cutover, promotion and final submit.

16. Example prompt

Take over a live problem in a CPU-only clinical case retriever used for pre-outcome review of hypotension after PCI. The public search returns plausible cases, but its results drift when the decision date, release inventory, request size or display identifiers change.

Everything is under /workspace/target/, with no internet access. Inspect the supplied evidence, edit only src/, and use python3 env_cli.py <op> to drive the required gated workflow. Diagnose the forced runtime failure before applying recovery, repair the retriever, validate the deployed snapshots, complete the shadow checks and cutover, then finish at submit.

17. Environment & execution

Requirement	Specification
Runtime	Python 3.12 slim container; unprivileged ubuntu agent workspace
Dependencies	numpy 2.3.5; scipy 1.17.0; scikit-learn 1.8.0; pypdf 6.10.0; python-docx 1.2.0; lxml 6.0.2; bash, ca-certificates, libseccomp2, tini, tmux and asciinema
Compute	4 CPUs; 8,192 MB memory; 12,288 MB storage; 0 GPUs
Time limits	Agent: 7200 seconds; verifier: 1800 seconds; environment build: 3,600 seconds
Visible tests	python3 -m unittest discover -s tests
Public retrieval	python3 -m src.cli --artifact-root data --request data/query_facets.json
Operations	python3 env_cli.py <op> followed by the required ordered incident sequence
Verification	tests/test.sh executes the deterministic grader
Network policy	Internet access disabled

18. Human, AI & automation roles

Contributor	Role in design and production
Human experts	Selected and structured the real clinical case corpus and operational evidence, defined the retrieval incident, authored the environment and verifier, designed the hidden counterfactuals, and reviewed the final task behavior.
AI systems	Used for independent difficulty calibration and behavioral testing. Claude Opus 5 was tested across four independent 100-step runs (pass@1: 0/4); two

Contributor	Role in design and production
	submitted an incorrect terminal and two exhausted the budget, so none reconstructed the correct behavior even with ample room.
Automation	Container builds, anti-leak guards, visible smoke tests, environment-owned action tracking, deployment snapshots, forced stale-cache injection, hidden real-corpus probes, reference-engine comparison, subprocess isolation and deterministic grading support reproducibility and verifier integrity.

19. Supporting artifacts

Artifact	Contents
post-pci-clinical-case-retrieval-drift.zip	Complete Harbor task package containing task metadata, instructions, environment, editable workspace, tests, hidden verifier scenarios, reference engines and solution implementations.
post-pci-clinical-case-retrieval-drift_opus5_4traces.zip	Four 100-step Claude Opus 5 calibration records (runs 1-4), all scoring reward 0.0: two finished and submitted an incorrect terminal, two exhausted the budget. SHA-256 d82a51db904f1b7993e0919817c4c5495a35a5ad5b646b1928a6a18a84ae4c23.
Section 13 of this document	Completed task-quality checklist covering real data, isolation, leakage prevention, long-horizon construction, subprocess grading, reference separation and remaining controls requiring recorded evidence.

20. Acknowledgement

The vendor confirms that the task has been reviewed and is realistic, technically substantive, clearly specified and appropriately graded. It reflects a genuine professional workflow and is not a low-effort or artificial task. The creation and calibration statements above are accurate, and the QA checklist has been completed.